601.433/633 Introduction to Algorithms **Fall 2026**
Homework #8 **Due:** November 12, 2026, 11:59pm

Remember: you may work in groups, but must write up your solution entirely on your own. Solutions must by typeset (LaTeX preferred but not required). Collaboration is limited to discussing the problems – you may not look at, compare, reuse, etc. any text from anyone else in the class. You may use the internet to look up formulas, definitions, etc., but may not simply look up the answers online.

Please include proofs with all of your answers, unless stated otherwise.

1 Boruvka's Algorithm (33 points)

Let $G = (V, E)$ be a connected, undirected graph and let $w : E \to \mathbb{R}_{\geq 0}$ be *distinct* edge weights (w is injective). We're going to analyze yet another MST algorithm: Boruvka's MST algorithm (from 1926), which is a bit like a distributed version of Kruskal.

We begin by having each vertex mark the lightest edge incident to it. (For instance, if the graph were a 4-cycle with edges of lengths 1, 3, 2, and 4 around the cycle, then two vertices would mark the "1" edge and the other two vertices will mark the "2" edge). This creates a forest F of marked edges. (Convince yourself that there won't be any cycles!). In the next step, each tree in F marks the shortest edge incident to it (the shortest edge having one endpoint in the tree and one endpoint not in the tree), creating a new forest F'. This process repeats until we have only one tree.

(a) (11 points) Show correctness of this algorithm by proving that the set of edges in the current forest is always contained in the MST.

We use a proof by induction via loop invariant on Boruvka's MST algorithm. We consider graph $G = (V, E)$ as described in the problem.

Base case. At the start, each $v \in V$ is a separate tree. Thus, there are 0 edges in the current forest. Since there are no edges in the current forest, every edge in the forest must be in the MST, proving the base case true.

Inductive step. We assume that for iterations 1 to $i - 1$ of Boruvka's, every edge added to forest F is in the MST for G. We prove that on iteration i, each edge in the new forest F' is in the MST for G.

We consider tree T in F. Boruvka's will select the lowest-weight edge $e = (u, v)$ such that $u \in T$ and $v \notin T$, meaning there will be no cycles. Therefore, we can consider edge e to be the minimum edge that crosses the cut $(T, \overline{T})$. We know, by the Inductive Hypothesis, that cut $(T, \overline{T})$ respects the MST. By **Theorem 17.2.6**, the lowest-weight edge e that crosses any cut respecting the MST is safe for the set of edges in a graph's MST. Thus, since e is the lowest-weight edge crossing $(T, \overline{T})$, then e is safe to add to the MST.

This process repeats for all trees in F. Since any cut respects the MST, any edge added between two trees in F is safe for the MST. Therefore, after iteration i of Boruvka's, every edge in the new F' is safe for G's MST.

In proving correctness for Boruvka's algorithm in the base and inductive case, we have proven correctness for Boruvka's algorithm as a whole.

(b) (11 points) Show how you can run each iteration of the algorithm in $O(m)$ time with just a few runs of DFS and no fancy data structures (heaps, union-find – remember, this algorithm was from 1926!). In other words, given a current forest F of marked edges, show how to find the set of edges which consists of the shortest edge incident to each tree in F in time $O(m)$.

For each iteration, we use DFS search to identify which vertices belong to which tree. We create a dictionary to keep track of, for each vertex v, which tree T_i within G such that $v \in T_i$. We run DFS on F, keeping track of which nodes are reachable from others. As per the notes in **Lecture 13**, we know that the time complexity of DFS is $O(n+m)$. However, we can simplify this further. Since G is a connected graph, we know that $m \geq n-1$, since the graph must contain at least enough edges to connect the n vertices. Rearranging, we get $m+1 \geq n$. Thus, $O(n) = O(m+1) = O(m)$. Inserting this into the DFS, we get that the time complexity of the DFS is $O(m)$.

From here, we initialize an array to keep track of the minimum outgoing edge from each tree T_i. In a graph with n vertices, there are at most n separate trees. Thus, this step takes $O(n) = O(m)$ time, as per the proof above.

Now that we know to which components each tree is in, we can now iterate through each edge in G. For each edge (u, v) in G, we check if $u \in T_i$ and $v \notin T_i$, and compare it against the current minimum outgoing edge weight from T_i. If (u, v) is smaller, it becomes the new minimum outgoing edge from T_i. Since this aspect of the iteration iterates through each edge in G once, we know it takes $O(m)$ time.

Once this round is done, we add each minimum edge to forest F. We add 1 edge for each T_i. Since this is a maximum of n trees, we know this takes $O(n) = O(m)$ time.

Summing this, we can see that each iteration of Boruvka's takes $O(m) + O(m) + O(m) + O(m) = O(m)$ time.

(c) (11 points) Prove an upper bound of $O(m \log n)$ on the running time of this algorithm.

The previous question showed that each iteration of takes $O(m)$ time. We will prove that there will be at most $\log n$ iterations of the algorithm, yielding overall time complexity $O(m) * \log n = O(m \log n)$.

We consider an arbitrary iteration of Boruvka's, where the forest F has k trees T in it. For each tree T_i, we select 1 outgoing edge from it to add to F. Since the edge is between 2 trees, adding it to the forest would join at least 2 edges. This means that every resulting tree in the new F' contains at least 2 of the trees in F. Therefore, the number of components k' is $k' \leq \frac{k}{2}$.

Thus, we know that each iteration at least halves the number of trees. We begin with n trees, so after i rounds, there are $k \leq \frac{n}{2^i}$ groups. Since the algorithm stops when the number of trees is 1, we know there must be 1 tree in F. We can calculate the number of rounds to reach 1 tree as:

$\frac{n}{2^i} = 1$

$n = 2^i$

$i = \log_2(n)$

Therefore, the number of iterations is $O(\log n)$. Since the amount of work per round is $O(m)$, we know the total time complexity is $O(\log n) \cdot O(m) = O(m \log n)$.

2 Clustering (33 points)

Consider the following problem. You are given a graph $G = (V, E)$ and a length function $\ell : E \to \mathbb{R}_{\geq 0}$. You should output a clustering (i.e., a partition) $(C_1, C_2, \ldots, C_k)$ of V into k clusters which maximizes the minimum distance between clusters. More formally, you should maximize $\min_{i,j \in [k], i \neq j} d(C_i, C_j)$, where $d(C_i, C_j) = \min_{x \in C_i, y \in C_j} d(x, y)$ and where $d(x, y)$ is the shortest-path distance (under length function ℓ). Design an algorithm for this problem which runs in $O(m \log n)$ time, and prove correctness and running time.

Hint: Think about MSTs and Kruskal's algorithm

Algorithm Description:

We begin with the same start to Kruskal's algorithm. We first sort the edges by weight ascending, so that the output is a list of the edges where the first edge is the lowest-weight edge. From here, we begin building a forest F. We use a Union-Find data structure to represent F. We repeatedly perform unions on F, adding the next lowest-weight edge, until there are k sets in the structure. We then return the forest F, representing the partition.

Correctness Proof:

We prove correctness by induction on k.

Base case. We must first address the inherent limitation on k. Since we are trying to produce a partition of G, the maximum number of components is n, where n is the number of nodes, since that means each node is in its own component. Thus, we consider the maximum possible value of k for our base case. Thus, $k = n$. This means that the resulting forest after the algorithm should produce n components. This means each node is its own component. There is only one partition where $k = n$, so this partition is the maximum of the minimum distance between components. Thus, the base case holds.

Inductive step. We assume that this algorithm returns the correct partition for G where $k = i + 1$. We will prove this holds for $k = i$. When $k = i + 1$, we know that forest F has $i + 1$ sets. To go from $k = i + 1$ to $k = i$, this means that 2 sets must be unioned to create 1 set. Our algorithm will select the lowest-weight edge that does not create a cycle to add to F. Any edge considered either would join 2 vertices already in the same connected component, creating a cycle (and therefore not valid to add to F), or it would join 2 vertices in different components, which unions those 2 sets into 1 set. Therefore, F now has i components.

Let w be the weight of the newest added edge x to F. Since w is the next lowest-weight edge, we know that all edges e not in F have weights $\ell(e) \geq w$. Therefore, the minimum possible distance between any of the remaining i clusters is exactly w.

Now, we assume FTSOC that there exists another partition F' of i components with a minimum distance between clusters greater than w. For this to happen, every edge with weight at most w, must have endpoints in the same cluster in this alternative partition. Otherwise, the minimum distance between two clusters would be at most w. Now, we consider the $i + 1$ components before adding x. Because all edges of weight at most w must have endpoints in the same alternative cluster, each of these $i + 1$ components must be entirely contained within one alternative cluster. However, there are only i alternate clusters. By the pigeonhole principle, at least two of the $i + 1$

components must belong to the same alternative cluster. Thus, this is a contradiction, so it is impossible for any other partition F' to have a minimum distance greater than w.

Therefore, the parition found by this algorithm is correct, so the inductive step holds.

In proving the base and inductive case correct for all valid k such trhat $1 \leq k \leq n$, we have proven the algorithm correct.

Runtime Proof:

As per the proof in **Theorem 17.4.1**, we know that sorting the edges in the tree takes $O(m \log m) = O(m \log n)$ time. From here, we initialize the Union-Find structure, with n nodes, which takes $O(n)$ time. We expect this algorithm to work for all $k \leq n$. Thus, the algorithm must work for $k = 1$. Therefore, G must be a connected graph, since it must have at least the number of edges to connect every node, or $n - 1$ egdes. Therefore $O(n) = O(m + 1) = O(m)$. Finally, we consider the maximum number of unions that can be done on the tree. If $k = 1$, then we are trying to produce 1 connected component. This is no different than running Kruskal's normally, so we know that the worst-case runtime of the unions is $O(m\alpha(m, n))$, per **Theorem 17.4.1**. Summing, we get a runtime of $O(m \log n) + O(m) + O(m\alpha(m, n))$. The $O(m \log n)$ term dominates, giving a total runtime of $O(m \log n)$.

3 Matroids (34 points)

(a) (17 points) Let U be a finite set and let $U_1, U_2, \ldots, U_k$ be a partition of U into nonempty disjoint subsets (where $k \geq 2$). Let $r_1, r_2, \ldots r_k$ be positive integers. Let $\mathcal{I} = \{S \subseteq U : |S \cap U_i| \leq r_i\}$ for all $i \in \{1, 2, \ldots k\}$. Prove that $(U, \mathcal{I})$ is a matroid.

To prove that $(U, \mathcal{I})$ is a matroid, we must prove the following proprties hold:

1. $\emptyset \in \mathcal{I}$
2. Let A, B be sets. If $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$
3. Let A, B be sets. If $A, B \in \mathcal{I}$ and $|A| < |B|$, then for some elements x such that $x \in B$ A, $A \cup \{x\} \in \mathcal{I}$

Property 1:

We consider the empty set $\emptyset$. $\emptyset$ contains no elements. Therefore it contains 0 elements from each set in the partition of U. Therefore, $\emptyset \in \mathcal{I}$ and this property holds.

Property 2:

We consider sets A and B, where $A \in \mathcal{I}$ and $B \subseteq A$. Thus, all elements of B are in A. Since $A \in \mathcal{I}$, for each i, A contains at most r_i elements from U_i. B contain no elements that are not also in A. Therefore, B cannot contain more than r_i elements in any U_i. Thus, $B \in \mathcal{I}$, so this property holds.

Property 3:

We consider sets A and B, where $A, B \in \mathcal{I}$ and $|A| < |B|$. We can write the size of sets A and B as:

$|A| = \sum_{i=k}^{k}(|A \cap U_i|)$

$|B| = \sum_{i=k}^{k}(|B \cap U_i|)$

Since $|A| < |B|$:

$\sum_{i=k}^{k}(|A \cap U_i|) < \sum_{i=k}^{k}(|B \cap U_i|)$

For this inequality to hold true, there must be some i for which:

$|A \cap U_i| < |B \cap U_i|$

Since $B \in \mathcal{I}$, we know that for this specific value i, B has no more than r_i elements in it from U_i. Therefore A has strictly fewer than r_i elements in it from U_i. We consider x to be an arbitrary element in U_i such that $x \in B$ A. Now, we consider set $A \cup \{x\}$. Since A has strictly fewer than r_i elements in it from U_i, adding 1 element from U_i (x) means that the resulting set $A \cup \{x\}$ has no more than r_i elements in it from U_i. Thus, $A \cup \{x\} \in \mathcal{I}$, and this property holds.

In proving these 3 properties, we have proven that $(U, \mathcal{I})$ is a matroid.

(b) (17 points) Let $(U, \mathcal{I})$ be a matroid, and let $w : U \to \mathbb{R}^+$ be an assignment of weights to elements. We know that the greedy algorithm will return a maximum-weight independent set. But suppose we are also given some $I \in \mathcal{I}$, and want to return the maximum-weight independent set *that contains I*. Modify the greedy algorithm to solve this problem, and prove that your algorithm is correct. You do not need to analyze running time, but your algorithm should need only a polynomial amount of time and calls to the independence oracle.

Let W be the maximum weight independent set on matroid $(U, \mathcal{I})$. Let W' be the maximum weight independent set on matroid $(U, \mathcal{I})$ containing some given set $I \in \mathcal{I}$. We will modify the greedy algorithm that currently finds W to now find W'.

In the original greedy algorithm, we do the following:

1. Initialize $W = \emptyset$.
2. Sort elements of U in decreasing order of weight.
3. Consider each element x in sorted order. If $W \cup \{x\} \in \mathcal{I}$, add x to W.
4. Return W.

We consider a modification where instead of beginning with initializing $W = \emptyset$, we initialize $W' = I$. Then, instead of sorting elements of U, we sort elements of U/I, since we've already added I to W'. We follow the rest of the algorithm the same way, and return W'.

We know that the original greedy algorithm is correct as per **Theorem 18.4.1**. We prove correctness for this modification.

Let $W' = x_1, x_2, \ldots, x_k$ be the elements added by our modified greedy algorithm, in the order they were added, and let $W^* = y_1, y_2, \ldots, y_k$ be an arbitrary maximum-weight independent set containing I, with its elements outside of I sorted in decreasing order of weight. Since all weights are positive, both W' and W^* are maximal independent sets. By the augmentation property, all maximal independent sets in a matroid have the same size. Therefore, both sets contain the same number of elements outside of I, so we can use the same indexing for both.

We will prove that $w(x_j) \geq w(y_j)$ for every j. This implies that W' has weight at least that of W^*, and therefore is optimal. Assume FTSOC that this is false, and let j be the smallest index for which $w(x_j) < w(y_j)$.

Let $F_1 = I \cup x_1, x_2, \ldots, x_{j-1}$ and $F_2 = I \cup y_1, y_2, \ldots, y_j$. Both F_1 and F_2 are independent sets, and $|F_2| > |F_1|$. Therefore, by the augmentation property, there is some element $y_z \in F_2$ F_1 such that $F_1 \cup y_z \in \mathcal{I}$.

Since the elements $y_1, \ldots, y_j$ are sorted in decreasing order of weight, we know that $w(y_z) \geq w(y_j) > w(x_j)$. However, this means that the greedy algorithm would have considered y_z before x_j. Since $F_1 \cup y_z$ is independent, and the set of elements already added by the algorithm when it considers y_z is a subset of F_1, the algorithm would have added y_z to its solution.

This is a contradiction, since $y_z \notin F_1$, and F_1 contains all elements of I and all elements added by the algorithm before x_j.

Therefore, no such index j exists, and $w(x_j) \geq w(y_j)$ for every j. It follows that $w(W') \geq w(W^*)$. Since W^* was an arbitrary maximum-weight independent set containing I, our modified greedy algorithm returns a maximum-weight independent set containing I, as desired.